Algorithms
(Decision-Making Process Based on Distance)

黃祥睿 (Xiang-Rui Huang)
國立澎湖科技大學 資訊工程學系 兼任講師
Adjunct Lecturer, Department of Computer Science and Information Engineering
National Penghu University of Science and Technology Penghu, Taiwan
國立陽明交通大學 智慧科學暨綠能學院 博士研究生 (Ph.D. student)
College of Artificial Intelligence, National Yang Ming Chiao Tung University

ICSL Intelligent Computing and Systems Lab
MITLab Multimedia Innovative Technology Lab
AICV Lab Artificial Intelligence & Computer Vision

Goal

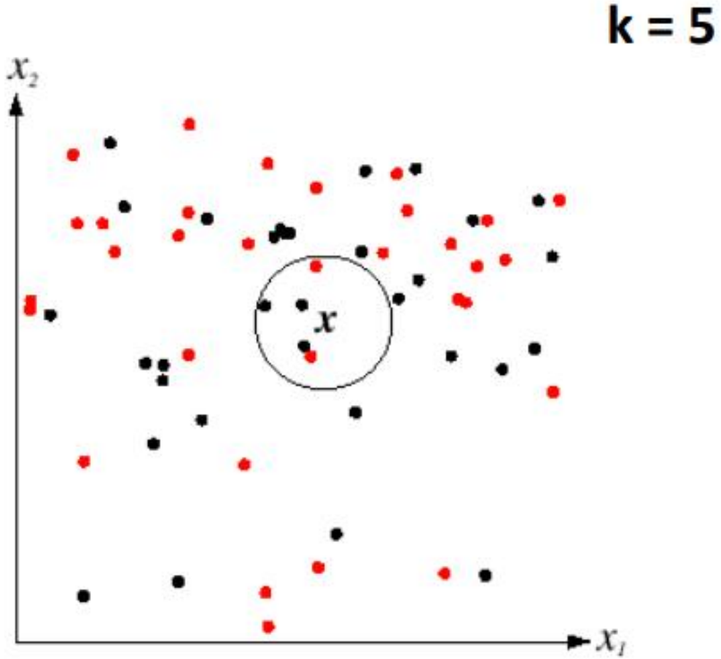
在決策樹中，我們透過一連串規則，將整個空間不斷切分，讓每一個區域都對應到一個明確的決策結果。換句話說，Tree 的核心思想是先建立結構，再依據這個結構做判斷。

然而，如果我們不想預先切分空間，而是讓每一個新的樣本自己去「觀察周圍」，會發生什麼事？這時候就會導向另一種完全不同的思維：不建立規則、不劃分區域，而是直接根據樣本所在的位置，尋找與它最接近的資料點，並依據這些鄰居的多數結果來做決策。這樣的方法，就是 KNN。

KNN

K-Nearest Neighbors

- For a new point, find the k closest points from training data
- Labels of the k points “vote” to classify
- Works well provided there is lots of data and the distance function is good



Source: D. Lowe

First classifier: Nearest Neighbor

```
def train(images, labels);
  # machine learning!
  return model
```

→ Memorize all data and labels

```
def predict(model, test_images);
  # Use model to predict labels
  return test_labels
```

→ Predict the label of the most similar training image

First classifier: Nearest Neighbor

```
def train(images, labels);
  # machine learning!
  return model
```

→ Memorize all data and labels

Training data with labels

```
def predict(model, test_images);
  # Use model to predict labels
  return test_labels
```

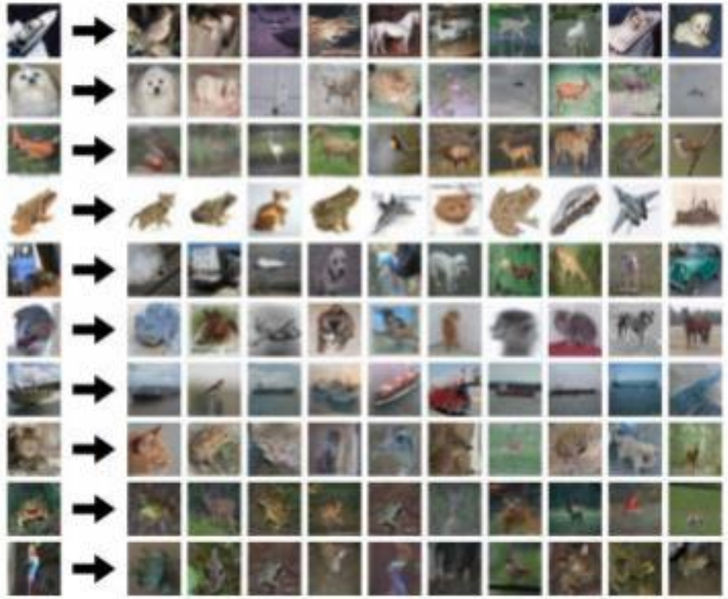
query data

→ Distance Metric $\left| \text{ }, \text{ } \right| \to \mathbb{R}$

Example Dataset: CIFAR10

10 classes
50,000 training images
10,000 testing images

Test images and nearest neighbors



Nearest Neighbor classifier

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

A. O(1) for training and O(1) for evaluation
B. O(1) for training and O(N) for evaluation
C. O(N) for training and O(1) for evaluation
D. O(N) for training and O(N) for evaluation

Nearest Neighbor classifier

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

A good implementation:
https://github.com/facebookresearch/faiss

Nearest Neighbor classifier

Q: With N examples, how fast are training and prediction?

Answer:
- O(1) for training
- O(N) for evaluation

This is bad: we want classifiers that are fast at prediction; slow for training is ok

Nearest Neighbor classifier

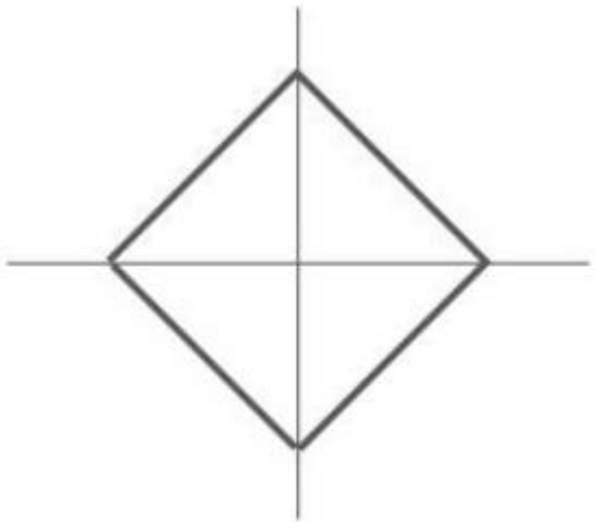
Instead of copying label from nearest neighbor, take majority vote from K closest points

K = 1 K = 3 K = 5

Nearest Neighbor classifier

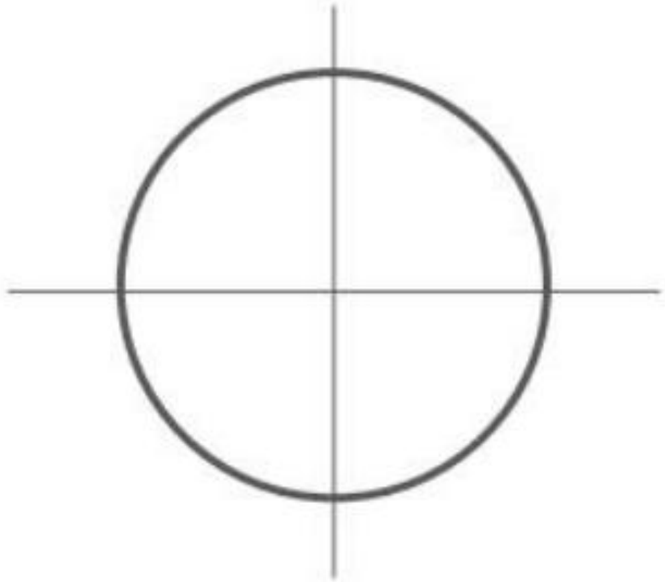
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p \left| I_1^p - I_2^p \right|$$



L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p \left(I_1^p - I_2^p \right)^2}$$



Nearest Neighbor classifier

K值	特性	問題
小（K=1）	很敏感	容易 overfitting
中（K=3~5）	平衡	通常最好
大（K很大）	很平滑	容易 underfitting

KNN主要問題

1.局部偏差（Local Bias）
只依賴附近資料
如果該區資料錯誤 會持續做出錯誤判斷
無法跳出錯誤區域

2.多數不等於正確（Majority ≠ Correct）
使用多數決
多數只代表最多人選
不代表最接近真相

3.過度平滑（Over-smoothing）
K 值過大
邊界變模糊
重要細節消失
容易 underfitting

4.高維災難（Curse of Dimensionality）
維度增加時
所有點距離變得相似
最近鄰失去意義

5.對雜訊敏感（Noise Sensitivity）
K 值過小（例如 K=1）
容易被單一錯誤資料影響
容易 overfitting

6.缺乏模型理解能力
不進行學習
不建立模型
無法解釋決策原因

Formalizing human decision-making

Lecture

為人之術：
在 AI 與演算法
文化中重思靈韻
The Art of Being Human:
Rethinking Aura in the Age of AI
and Algorithmic Culture
00:00:12
01:45:55
Video Project 7
國立嘉義大學外語系黃柏源副教授

演算法

Assignment 1 (2026/4/8)

Assignment – See Class Lecture

Assignment : See Class Lecture

- **Please share your reflections.**
- 學生姓名，學號
- 講者介紹（約200字）
- 關鍵字五個
- 演講內容摘要（中文 1000 字以上）
- 提問或省思（中文 1000 字以上）

Assignment 2 (2026/4/8)

海龜湯:沉默的選擇
從眾效應（Conformity / Asch Experiment）

這座城市有一條街，被監視器稱為「最後的路口」。所有失蹤的人，都曾在那裡停下來幾秒。他們沒有爭扎，沒有猶豫，只是安靜地看向四周的人，然後像是得到某種默契般，同時轉身，走進同一條巷子。

奇怪的是，那條巷子什麼都沒有，警方搜過無數次，空無一物。唯一的線索，是這些人在做決定前，都會觀察「附近的人怎麼走」，而且人越多，他們越相信那個方向。直到唯一的倖存者顫抖著說出真相那天他差點也跟著走，但他突然發現，周圍所有人其實都在看彼此，沒有人真正知道要去哪裡。

當他意識到這一點時，他停下了腳步。幾秒後，那些人已經全部消失在巷子裡。那一刻他才明白，最可怕的不是那條路，而是當所有人都依賴彼此做選擇時，錯誤會變成唯一的方向。

沉默的選擇

KNN 城市模擬懸疑 Challenge

一座城市，三條出口，所有人都走同一條路。
第一晚你跟著最近的居民，平安無事；
第二晚你相信更多人，差點消失。
問題不是「哪條路最近」，而是「當鄰居不可靠時，你還能相信誰？」

霧城模擬 | 多數決陷阱 | 改造 KNN

目標：在城市模擬中重現這場驚悚決策，再設計一個 KNN 生存策略。

Algorithm Culture · Trust · Bias · Survival

劇情設定：霧城的三夜

不是推理誰殺了人，而是推理：為什麼「同一套決策規則」會把你帶向不同命運。

第一夜

你只看最近的幾位居民。
你跟著他們走，安全抵達出口。

K 小：局部決策，暫時有效

第二夜

你擴大觀察範圍，相信更多人。
人群更一致，但你差點被帶進錯誤區。

K 變大：多數決開始放大偏差

第三夜

你發現南區居民彼此模仿。
他們彼此很近，卻一起走錯。

近，不一定可靠

23:50
十字路口

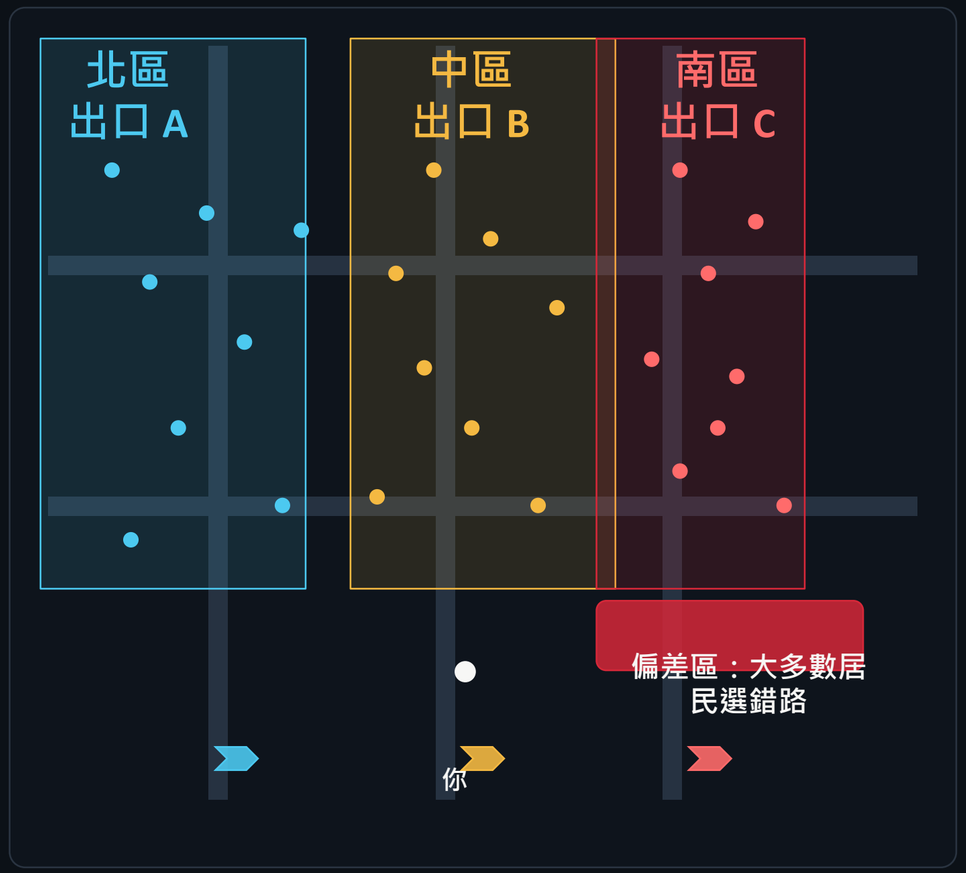
城市只剩路燈與腳步聲。
三個出口寫著同樣一句話：
「跟著大多數的人，就能活下來。」

但第二晚，你差點消失。

這個 Challenge 到底要做什麼？

你不是只要講故事；你要把故事「模擬出來」。

A. 建立城市模擬



B. 任務說明

- 建立一座至少 3 區的城市：北區 / 中區 / 南區，各區居民分布不同。
- 每位居民每天根據最近的 K 位鄰居，決定隔天走哪個出口。
- 刻意設計一個「偏差區」：那裡的居民彼此很近，卻集體選錯路。
- 放入一位調查員（你），測試在不同 K 下會不會被錯誤多數誤導。

C. 你要交什麼

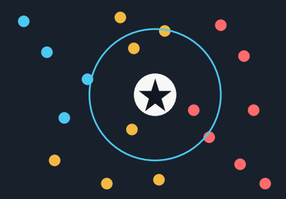
- 城市模擬示意圖或動畫截圖
- K = 1 / 3 / 5 的結果比較
- 一個你提出的 KNN 改造策略
- 300–500 字說明：為什麼你的方法比較不容易被誤導

Baseline：先讓 KNN 失敗一次

先用傳統 KNN 重現災難，再思考你要怎麼救它。 $\hat{y} = \text{majority}(N_k(x))$

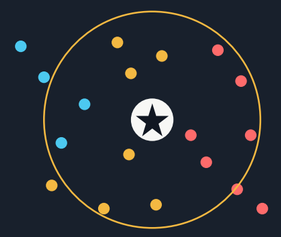
預設規則
看最近的 K 個鄰居 → 多數決 → 選出口

K = 1



K 很小：容易被局部雜訊牽走

K = 3



K 中等：有時穩定，有時剛好踩中偏差區

K = 5

K 很大：偏差區共識被放大，整體被誤導

請在你的模擬中，一眼看出：為什麼第二夜會失敗。

不是因為他不理性，而是因為他把「距離」誤當成了「可信度」。

真正的 Challenge：你要怎麼改 KNN？

請至少提出一種策略，讓調查員在偏差區附近仍有機會活下來。

距離加權

Weighted KNN

越近的鄰居影響越大。
適合局部結構還可信的情況。

你可以直接實作，或提出自己的變形版本。

自適應 K

Adaptive K

密集區用大 K，
混亂區用小 K。

你可以直接實作，或提出自己的變形版本。

可信度建模

Trust-aware KNN

不是每個鄰居都一樣可靠。
你可以為居民加上 trust 分數。

你可以直接實作，或提出自己的變形版本。

拒絕不穩定區

Reject / Abstain

如果鄰居彼此矛盾，
模型可選擇暫不跟隨。

你可以直接實作，或提出自己的變形版本。

最後請回答這句話：

如果你知道周圍的人都錯了，你還會跟隨他們嗎？

你的方法，必須把這句話變成一個可以在城市模擬裡驗證的演算法。

評分重點

劇情重現 你的模擬是否真的重現「第二夜差點消失」的感覺？

KNN 理解 你是否清楚展示 K 與偏差區之間的關係？

方法設計 你的改法是否比 baseline 更合理、更能說服人？

敘事表達 觀眾看完能不能立刻知道：你為什麼不再相信多數？

這不是在比誰最會寫 KNN。

這是在比：誰能在一個充滿錯誤共識的城市裡，
設計出仍然敢於懷疑、敢於不跟、敢於活下來的決策。

提交格式建議

- 6–8 頁簡報或 3 分鐘展示影片
- 附城市模擬圖、baseline 結果、改造方法
- 最後一頁必答：你的方法為什麼值得相信？

沉默的選擇

Challenge Deck · KNN × 城市模擬 × 演算法文化

WHY SAME RULE → DIFFERENT OUTCOME?

- 當你做決策時：
- 你要聽多少人？
- 何時該相信自己？

Assignment 3 (2026/4/8)

楚門的世界

如果世界是錯的，你怎麼做決策？

挑戰：當鄰居都是錯的

- 如果大多數鄰居都是錯的呢？
- 多數決還可靠嗎？
- KNN 還能相信嗎？

KNN 的隱含假設

- 鄰居是可靠的
- 局部區域代表真實分布
- 距離 ≈ 相似 ≈ 正確

關鍵問題

- 距離真的等於可信度嗎？
- 靠近的資料可能是錯的嗎？
- 如果局部區域本身有偏差呢？

最終思考

- 如果你知道周圍的人都錯了，
- 你還會跟隨他們嗎？
- 什麼時候應該相信自己？

THANK,
FOR WATCHING!